CS & IT ENGINEERING



Algorithms

Heap Algorithms
Discussion Notes

DPP – 01



By- Aditya sir

[MCQ]

#Q. Which of the following is valid min heap?

A. 34, 38, 35, 39, 40, 36, 50 → valid

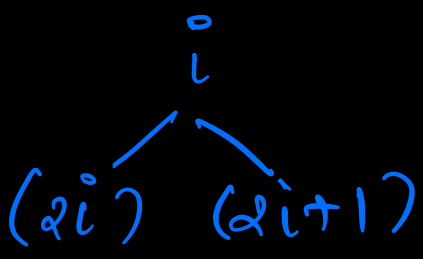
B. 25, 14, 12, 13, 10, 8, 16 ✗

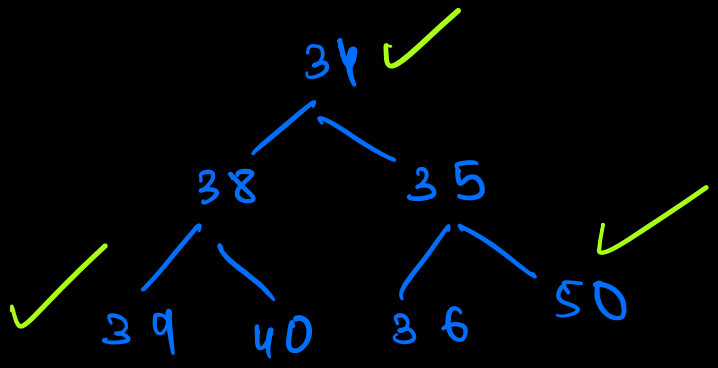
C. Both (a) and (b) ✗

D. Neither (a) nor (b) ✗

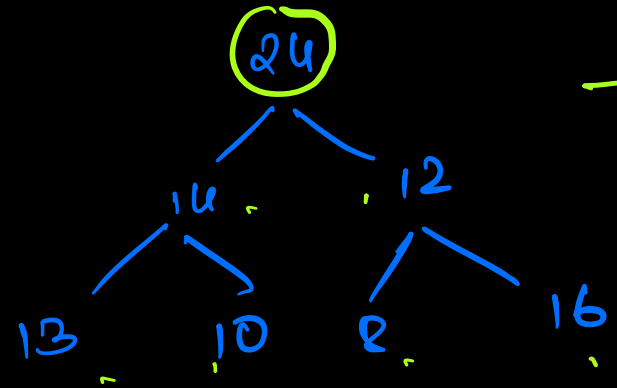
Ans : A

A) 34 38 35 39 40 36 50





B) 24 14 12 13 10 8 16



→ Invalid min-Heap.

[MCQ]

#Q. The minimum number of interchanges needed to convert the array into a max-heap is

89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70

A. 0

B. 1

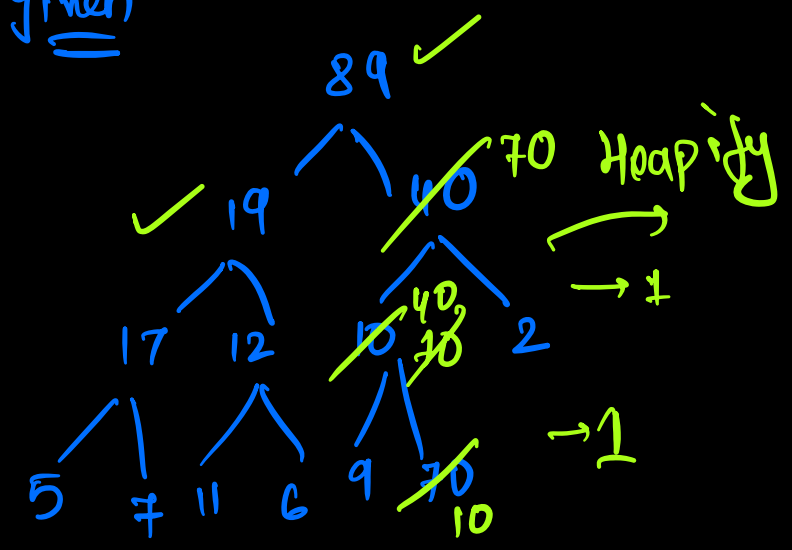
C. 2

D. 3

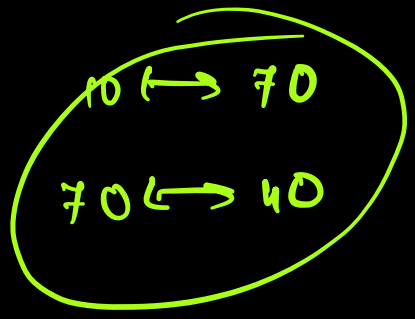
Ans :- C

89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70

Given



Total 2 swaps



[MCQ]

#Q. Consider a complete binary tree where the left and right subtrees of the root are max-heaps.
The lower bounds for the number of operations to convert the tree to a heap is

↳ min

A $\Omega(\log n)$ ✗

B $\Omega(1)$ ✓

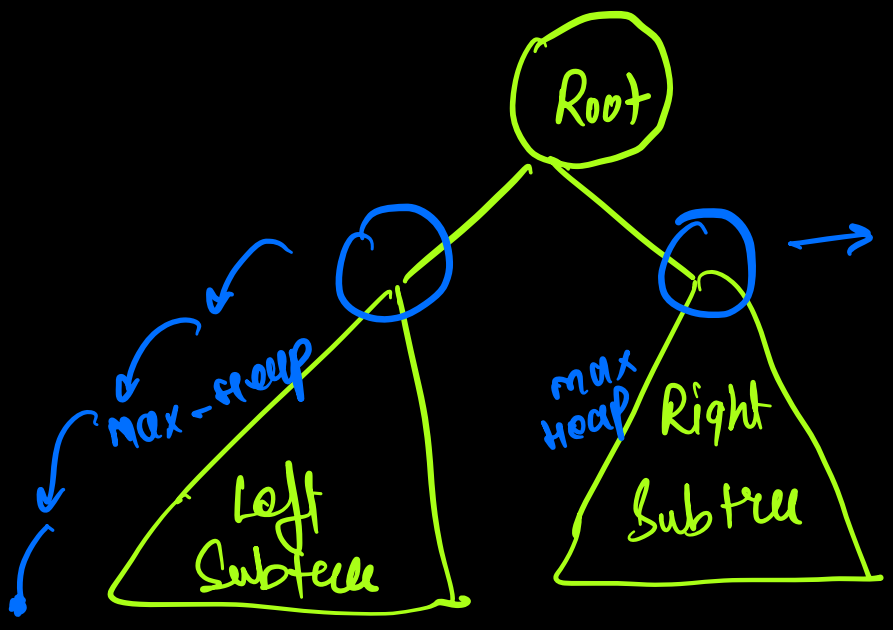
C $\Omega(n \log n)$ ✗

D $\Omega(n^2)$ ✗

Heap → By default Max-Heap

$\Omega(1)$ ✓

$O(\log n)$ ✓



→ Already a max-heap

1 comparison

Constant time

O(1)

Max Operations = height ⇒ O(log n)

[MCQ]

#Q. Consider a max heap represented by the array :
40, 30, 20, 10, 15, 16, 17, 8, 4

Array	1	2	3	4	5	6	7	8	9
value	40	30	20	10	15	16	17	8	4

Now consider that a value 35 is inserted into the heap. After insertion new heap is : -

A. 40, 30, 20, 10, 15, 16, 17, 8, 4, 35

B. 40, 35, 20, 10, 30, 16, 17, 8, 4, 15

C. 40, 30, 20, 10, 35, 16, 17, 8, 4, 15

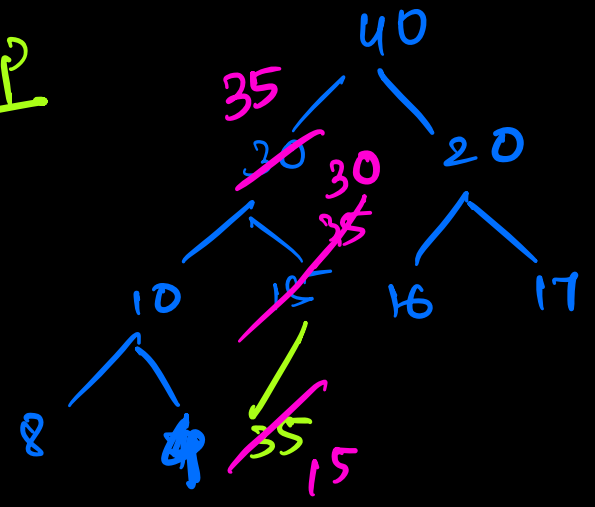
D. 40, 35, 20, 10, 15, 16, 17, 8, 4, 30

Ans: (5)

Heap → Insert operation.

40, 30, 20, 10, 15, 16, 17, 8, 7

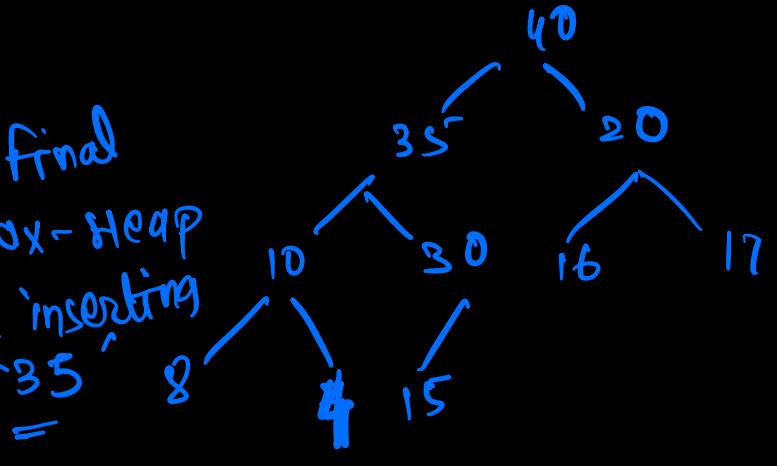
max-heap

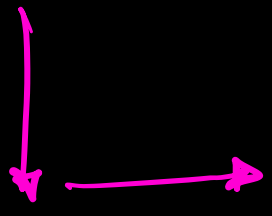


Insert (35)

→ Complete Binary Tree logic

Final max-heap after inserting 35





40, 35, 20, 10, 30, 16, 17, 8, 4, 15

[NAT]

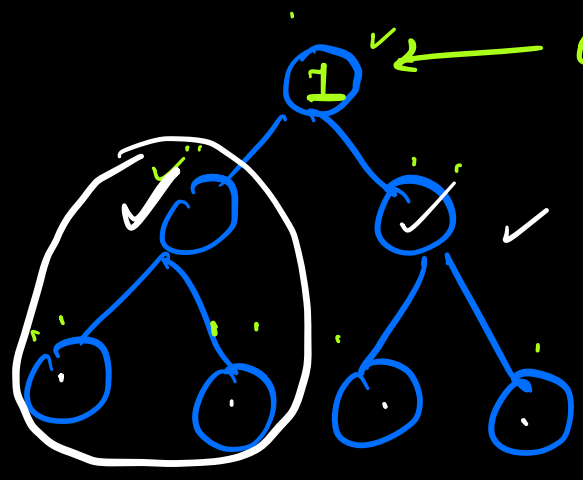
#Q. The number of possible min -heap containing each value from {1, 2, 3, 4, 5, 6, 7} exactly once.

1 2 3 4 5 6 7

Ans :- 80

Structure of Min-Heap with 7 elements

(1) 7



← overall minimum element.

~~1~~ 2 3 4 5 6 7

App 1 :-

$^6C_3 * 2 * 2$

$= \frac{6!}{(6-3)! \times 3!} \times 2 \times 2$

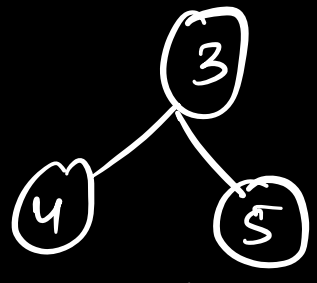
$= \frac{6!}{3! \times 3!} \times 4$

$= \frac{6 \times 5 \times 4 \times 3!}{3! \times 3 \times 2 \times 1} \times 4$

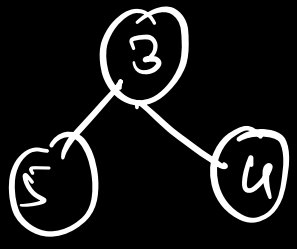
$= 5 \times 4 \times 4$

$= 20 \times 4 = \boxed{80}$

eg:- 3, 4, 5



or



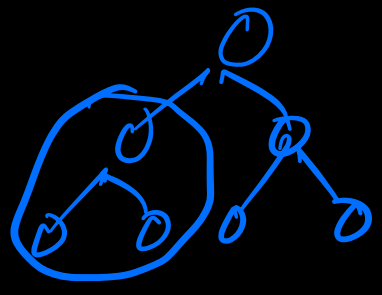
Appr 2 :- Formula

$$T(n) = {}^{n-1}C_k * T(k) * T(n-k-1)$$

where n = no. of elements to form Heap with.

k = no. of elements in left subtree.

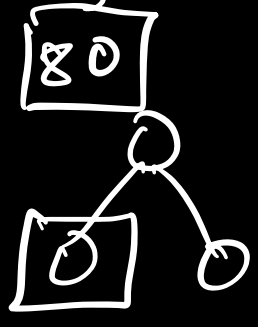
In our case: $n = 7$, $k = 3$



$$T(7) = {}^{7-1}C_3 * T(3) * T(7-3-1)$$

$$= {}^{6}C_3 * \frac{T(3)}{2} * \frac{T(3)}{2} = {}^{6}C_3 * 2 * 2$$

$= 80$



$T(3)$: $n = 3$, $k = 1$

$$T(3) = {}^{2}C_1 * T(1) * T(1)$$

$T(1) = 1$ (Base condition)

$T(3) = 2 * 1 * 1 = 2$

[MCQ]

Ans :- C

#Q. Consider the following statements : -

I. The smallest element in a max heap is always at a leaf node.

II. The second largest element in a max heap is always a child of the root node.

III. A max-heap can be constructed from a binary search tree in (n) time.

IV. A binary search tree can be constructed from a max-heap in (n) time.

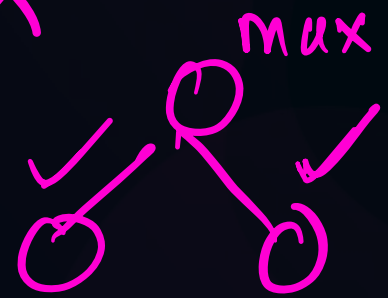
Which of the above statements are TRUE?

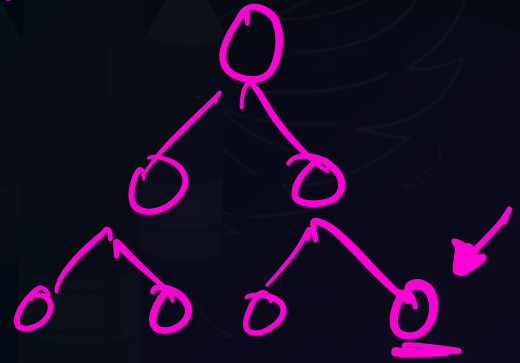
A. I, III and IV

B. II, III and IV

C. I, II and III

D. I, II and IV





[MCQ]

#Q. Let H be a binary min-heap consisting of n elements implemented as an array. What is the worst case time complexity of an optimal algorithm to find the maximum element in H?

A. $\Theta(\log n)$

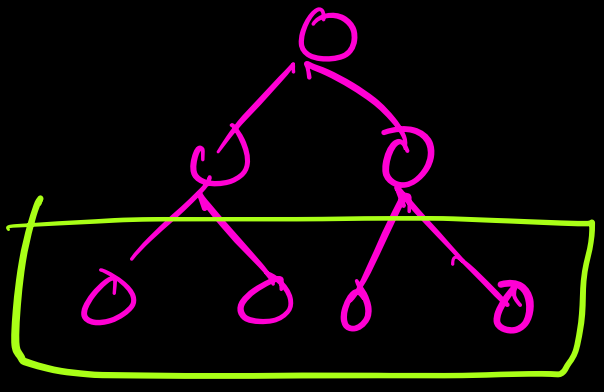
B. $\Theta(n \log n)$

C. $\Theta(n)$

D. $\Theta(1)$

Approach 1 :- Logic of Min-Heap.

$n = 7$



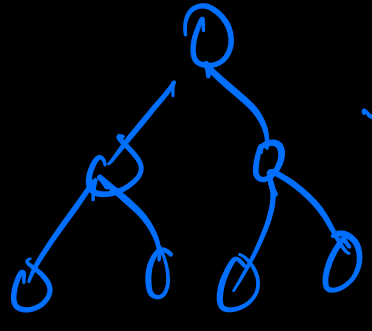
→ leaf level
⇒ 4 elems

Logic ≃ $O(n/2)$ elem are at leaf level.

Max elem will always be a leaf node in a Min-Heap.

→ $O(n)$ ✓

Approach2 :-



$O(n)$

Min Heap → Max Heap

$O(n)$

Max elem is always at root of max heap.

Heap using Heapify or Build Heap.

[MSQ] → multiple can be correct.

#Q. Which of the following is/are correct?

A. Bubble sort is stable but not inplace sorting technique

B. Insertion sort is stable sorting technique.

C. Selection sort is a stable sorting technique.

D. Bubble sort is a inplace sorting technique

Ans: B, C, D

Insertion Sort
Stable ✓
inplace ✓

Bubble Sort
Stable ✓
Inplace ✓

Stable

↓

Relative order of equal elems should be maintained before and after sorting.

inplace

Space Complexity at max

→ O(1) (excluding Rec stack)

n

→ O(log n) for Recursion stack.

Selection Sort

Stable ✓

inplace ✓

[MCQ]

#Q. Consider the following array

A	70	60	20	50	40	5	19	21

To sort this array using selection sort what is the result after 3rd pass?

A. 70, 60, 50, 40, 20, 5, 19, 21

B. 5, 19, 20, 21, 40, 70, 60, 50

C. 5, 20, 19, 50, 40, 70, 60, 21

D. 5, 19, 20, 50, 40, 70, 60, 21

Ans :- D

Soln:-
input A :
70 60 20 50 40 5 19 21
pass1 :
5 60 20 50 40 70 19 21
pass2 :
5 19 20 50 40 70 60 21
pass 3 :
5 19 20 50 40 70 60 21
pass 4 :
5 19 20 21 40 70 60 50
pass 5 :
5 19 20 21 40 70 60 50
pass 6 :
5 19 20 21 40 50 60 70
pass 7 :
5 19 20 21 40 50 60 70
final sorted output.

[NAT]

#Q. Consider the following array:

50	70	13	9	85	21	16

How many inversions are possible for above array A?

i, j such that

$i < j$ & $A[i] > A[j]$

Ans :- 12

	1	2	3	4	5	6	7
A:	50	70	13	9	85	21	16

(1,3) (1,4) (1,6) (1,7) → 4

(2,3) (2,4) (2,6) (2,7) → 4

(3,4) → 1

(4 → 0

(5,6) (5,7) → 2

(6,7) → 1

Total inversions = 4+4+1+0+2+1

= 12

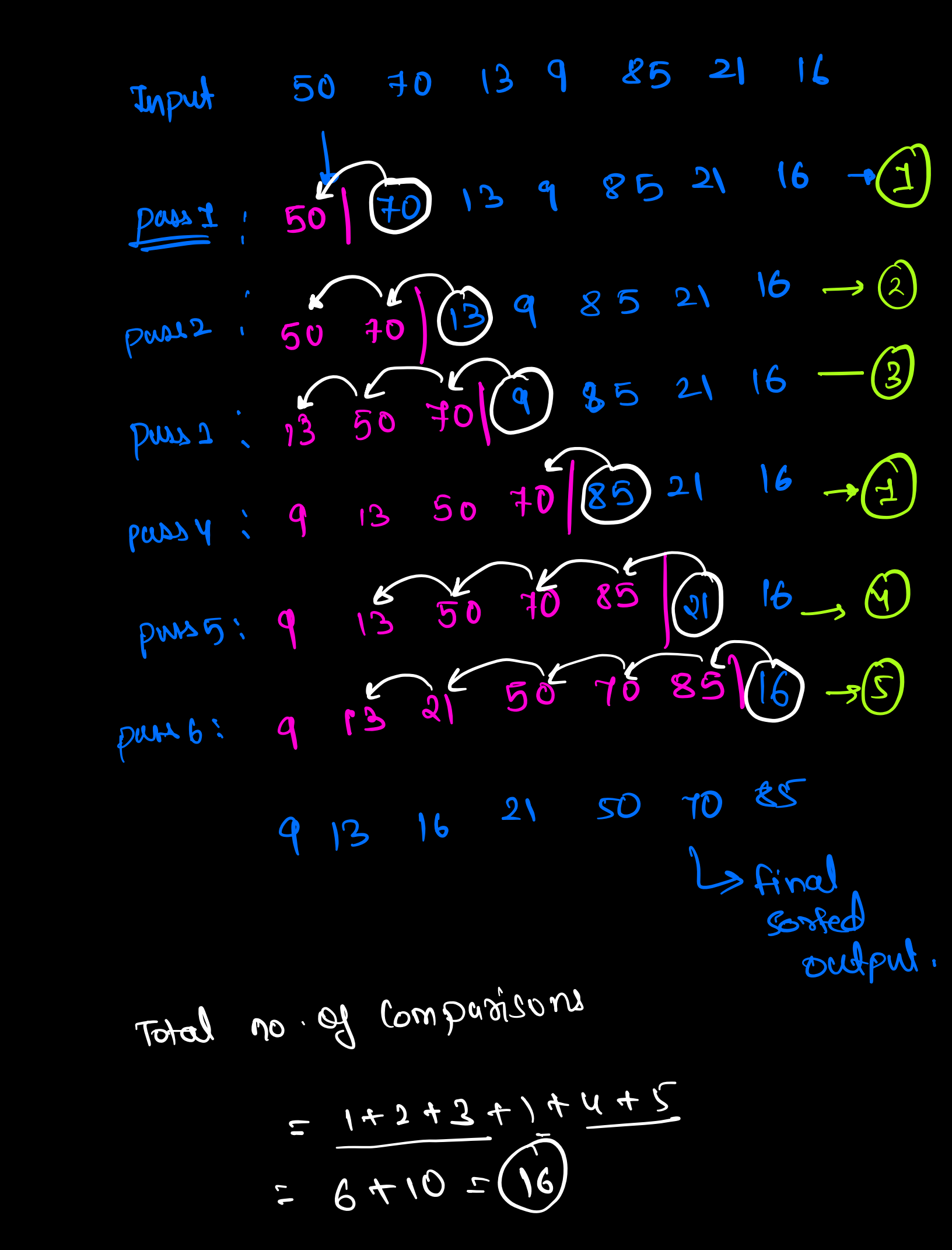
[NAT]

#Q. Consider the following array:

A	50	70	13	9	85	21	16

How many comparisons are needed to sort the above array using insertion sort?

Ans :- 16



Total no. of comparisons

$$= \frac{1+2+3+1+4+5}{}$$

$$= 6+10 = 16$$



THANK - YOU